# Apuntes teóricos · Clase 1 — Fundamentos y entrenamiento de redes neuronales

Oct 15, 2026 · @Andres O

Neural Networks: Deep Learning · Maestría en Ciencia de Datos y Máquinas de Aprendizaje (mención IA) · UIDE. Estos apuntes acompañan las sesiones del 23 y 24 de octubre y los notebooks NB01–NB03. Notación: vectores en negrita (**x**), matrices en mayúscula (W), superíndice entre paréntesis para la capa ($W^{(l)}$).

## 0 · Desde cero: IA, machine learning y deep learning

La inteligencia artificial (IA) es el campo que construye máquinas capaces de hacer tareas que asociamos con inteligencia, ya sea con reglas escritas a mano o aprendiendo. El machine learning (ML) es la parte de la IA en la que la máquina **aprende patrones de los datos** en lugar de seguir reglas escritas. El deep learning (DL) es la parte del ML que usa **redes neuronales de muchas capas**, capaces de descubrir por sí solas qué características mirar. Todo DL es ML y todo ML es IA, pero no al revés: un programa que rechaza todo crédito mayor a 1 000 dólares es IA en sentido amplio, pero no ML, porque no aprendió nada de los datos.

**Reglas o ejemplos.** En la programación tradicional una persona escribe reglas y el programa las aplica a los datos. En ML damos datos con sus respuestas correctas y el algoritmo encuentra las reglas (el modelo). Un filtro de spam con la regla “si dice GRATIS, es spam” falla cuando el estafador escribe GR4TIS; un filtro entrenado con miles de correos marcados aprende patrones más robustos.

| Tipo de aprendizaje | Qué recibe | Ejemplo |
|---|---|---|
| Supervisado (esta semana) | Datos con la respuesta correcta | Predecir si un socio cae en mora |
| No supervisado | Datos sin respuesta | Agrupar clientes por hábitos de compra |
| Por refuerzo | Premios y castigos por sus acciones | Un agente que aprende a jugar un videojuego |

**Por qué ahora.** Las redes neuronales existen desde 1958, pero despegaron después de 2012 por tres factores juntos: muchos más datos, procesadores gráficos (GPU) que hacen millones de multiplicaciones en paralelo, y mejores técnicas de entrenamiento (ReLU, buenas inicializaciones, Adam).

**Cuándo conviene el DL.** Brilla con imágenes, texto, audio y series largas, donde es difícil describir a mano qué mirar en los datos. En datos de tabla, como el mini-proyecto, una regresión logística o un modelo de árboles a veces lo iguala; por eso siempre se compara contra una línea base.

## 1 · La neurona artificial

Una neurona calcula una suma ponderada de sus entradas, le suma un sesgo y aplica una función de activación. Para una entrada $\mathbf{x} \in \mathbb{R}^d$, pesos $\mathbf{w} \in \mathbb{R}^d$ y sesgo $b \in \mathbb{R}$:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b, \qquad a = \varphi(z)$$

z se llama preactivación y a, activación. El sesgo puede verse como un peso más cuya entrada vale siempre 1: con $\tilde{x} = (\mathbf{x}, 1)$ y $\tilde{w} = (\mathbf{w}, b)$ queda $z = \tilde{w}^\top \tilde{x}$.

**Analogía: el comité de crédito.** El comité de una cooperativa mira ingreso, antigüedad y atrasos de un socio ($\mathbf{x}$); confía más o menos en cada señal ($\mathbf{w}$, positivo a favor y negativo en contra); tiene una disposición general (b, negativo si es conservador); suma la evidencia (z) y decide ($\varphi$). Aprender es ajustar esa confianza después de ver quién pagó.

**Geometría.** El conjunto $H = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ es un hiperplano. Si $\mathbf{x}$ y $\mathbf{x}'$ están en H, restando sus ecuaciones queda $\mathbf{w}^\top(\mathbf{x} - \mathbf{x}') = 0$, así que $\mathbf{w}$ es perpendicular a H. La distancia con signo de un punto a H es:

$$d(\mathbf{x}) = \frac{\mathbf{w}^\top \mathbf{x} + b}{\lVert \mathbf{w} \rVert}$$

Multiplicar ($\mathbf{w}$, b) por $c > 0$ no mueve la frontera, pero hace más empinada la sigmoide: el modelo se vuelve más "seguro" sin cambiar su decisión.

**Ejemplo numérico (NB01, sección 1).** Con $\mathbf{x} = (0.8, 0.5, -1.0)$, $\mathbf{w} = (1.2, 0.7, -2.0)$ y $b = -0.5$: $z = 0.96 + 0.35 + 2.00 - 0.50 = 2.81$; el escalón da 1 (aprueba) y $\sigma(2.81) \approx 0.943$. El aporte mayor (+2.00) viene de tener menos atrasos que el promedio: peso negativo por variable negativa.

# 2 · Funciones de activación y no-linealidad

La activación decide qué sale de la neurona, y su derivada decide cuánto se puede aprender a través de ella.

| Función | φ(z) | φ′(z) | Rango | Uso típico |
|---|---|---|---|---|
| Escalón | 1 si z ⩾ 0, 0 si no | 0 (casi en todo punto) | {0, 1} | Perceptrón clásico; no sirve para gradiente |
| Sigmoide | $\sigma(z) = 1 / (1 + e^{-z})$ | $\sigma(z)(1 - \sigma(z)) \leqslant 1/4$ | (0, 1) | Salida de clasificación binaria |
| Tanh | tanh z | $1 - \tanh^2 z \leqslant 1$ | (−1, 1) | Capas ocultas centradas en 0 |
| ReLU | max(0, z) | 1 si z > 0, 0 si z < 0 | [0, ∞) | Capas ocultas (por defecto) |
| Softmax | $e^{z_i} / \Sigma_k e^{z_k}$ | $\partial a_i/\partial z_j = a_i(\delta_{ij} - a_j)$ | simplex | Salida multiclase |

**Derivada de la sigmoide.** Como $\sigma(z) = (1 + e^{-z})^{-1}$:

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\,\frac{e^{-z}}{1+e^{-z}} = \sigma(z)\,\bigl(1-\sigma(z)\bigr)$$

Su máximo es 1/4 en z = 0. Al encadenar capas sigmoides, el gradiente se multiplica por factores ⩽ 1/4: es el origen del desvanecimiento del gradiente (sección 9).

**Por qué hace falta no-linealidad.** La composición de funciones afines es afín:

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\,\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2) = W'\mathbf{x} + \mathbf{b}'$$

Una red de 50 capas sin activaciones equivale a una sola capa lineal: sigue trazando un hiperplano. Analogía: apilar diez lupas no da un microscopio electrónico, sigue siendo una ampliación.

# 3 · El perceptrón de Rosenblatt y el teorema de Novikoff

El perceptrón aprende corrigiendo solo cuando se equivoca, y si los datos son separables con margen, termina en un número finito de errores.

Con etiquetas $y \in \{-1, +1\}$, entrada aumentada $\tilde{x} = (\mathbf{x}, 1)$ y predicción $\hat{y} = \text{sign}(\mathbf{w}^\top \tilde{x})$, el algoritmo recorre los datos y, cada vez que $y_i\, \mathbf{w}^\top \tilde{x}_i \leqslant 0$, actualiza:

```
\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \tilde{\mathbf{x}}_i
```

Tras la corrección, $y_i\, \mathbf{w}'^\top \tilde{x}_i = y_i\, \mathbf{w}^\top \tilde{x}_i + \eta \|\tilde{x}_i\|^2$: el margen de ese ejemplo sube (usando $y_i^2 = 1$).

**Teorema (Novikoff, 1962).** Si existe $\mathbf{w}^*$ con $\|\mathbf{w}^*\| = 1$ tal que $y_i\, \mathbf{w}^{*\top} \tilde{x}_i \geqslant \gamma > 0$ para todo $i$, y $\|\tilde{x}_i\| \leqslant R$, el perceptrón con $\eta = 1$ y $\mathbf{w}_0 = 0$ comete a lo sumo $(R/\gamma)^2$ errores.

**Prueba.** Sea $\mathbf{w}_k$ el vector tras $k$ errores.

1. Cada error acerca $\mathbf{w}$ a $\mathbf{w}^*$: $\mathbf{w}_{k+1}^\top \mathbf{w}^* = \mathbf{w}_k^\top \mathbf{w}^* + y_i\, \tilde{x}_i^\top \mathbf{w}^* \geqslant \mathbf{w}_k^\top \mathbf{w}^* + \gamma$. Por inducción, $\mathbf{w}_k^\top \mathbf{w}^* \geqslant k\gamma$.
2. La norma crece despacio: $\|\mathbf{w}_{k+1}\|^2 = \|\mathbf{w}_k\|^2 + 2y_i\, \mathbf{w}_k^\top \tilde{x}_i + \|\tilde{x}_i\|^2 \leqslant \|\mathbf{w}_k\|^2 + R^2$, porque en un error $2y_i\, \mathbf{w}_k^\top \tilde{x}_i \leqslant 0$. Por inducción, $\|\mathbf{w}_k\| \leqslant \sqrt{k}\, R$.
3. Por Cauchy–Schwarz, $k\gamma \leqslant \mathbf{w}_k^\top \mathbf{w}^* \leqslant \|\mathbf{w}_k\|\, \|\mathbf{w}^*\| \leqslant \sqrt{k}\, R$, de donde $k \leqslant (R/\gamma)^2$. ∎

En el NB01 (sección 4) el perceptrón comete 2 errores en la primera época y ninguno en la segunda: con datos separables, el final está garantizado.

**El límite: XOR.** Ningún hiperplano separa (0,0), (1,1) → 0 de (0,1), (1,0) → 1. Si existiera, tendríamos $b < 0$, $w_2 + b \geqslant 0$ y $w_1 + b \geqslant 0$; sumando las dos últimas, $w_1 + w_2 + b \geqslant -b > 0$, lo que contradice $w_1 + w_2 + b < 0$ para (1,1). Minsky y Papert (1969) popularizaron este límite. Cuando los datos no son separables (casi siempre en la práctica), el perceptrón nunca deja de corregir; la variante *pocket* guarda el mejor $\mathbf{w}$ visto.

## 4 · Funciones de pérdida desde máxima verosimilitud

La pérdida no se inventa: es la log-verosimilitud negativa de un modelo probabilístico de los datos.

**Clasificación binaria.** Suponemos $Y \mid \mathbf{x} \sim \text{Bernoulli}(\hat{y})$ con $\hat{y} = \sigma(z)$. La verosimilitud de un dato es $p(y \mid \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y}$. Con $N$ datos independientes, maximizar el producto equivale a minimizar el promedio de su logaritmo negativo, la entropía cruzada binaria (BCE):

```
\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\Bigl[\,y_i\log\hat y_i
+ (1-y_i)\log(1-\hat y_i)\Bigr]
```

**Su gradiente respecto a $z$** (un dato) se simplifica de forma notable:

```
\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}
{\partial \hat y}\,\frac{\partial \hat y}{\partial z} = \left(-\frac{y}{\hat
y} + \frac{1-y}{1-\hat y}\right)\hat y(1-\hat y) = \hat y - y
```

Con MSE, ½(ŷ − y)², queda en cambio (ŷ − y)·σ′(z): si la neurona está saturada y equivocada, σ′(z) ≈ 0 y casi no aprende. En z = −6 con y = 1, el gradiente de BCE vale ≈ 0.998 y el de MSE ≈ 0.0025.

**Analogía: el pronóstico de Quito.** Si el pronóstico dijo "99 % de sol" y llovió, la BCE castiga con −log 0.01 ≈ 4.6; el MSE apenas se inmuta. La entropía cruzada mide sorpresa.

**Regresión.** Si Y | **x** ~ N(ŷ, s²) con s fija, la log-verosimilitud negativa es, salvo constantes, (y − ŷ)²/(2s²): el MSE.

**Multiclase.** Con K clases, ŷ = softmax(**z**) y etiqueta one-hot **y**, la entropía cruzada categórica es $-\Sigma_k\, y_k \log \hat y_k$ y su gradiente vuelve a ser $\partial\mathcal{L}/\partial\mathbf{z} = \hat y - \mathbf{y}$.

**Detalle práctico.** Con 12.7 % de mora, un modelo que predice la prevalencia para todos tiene BCE ≈ −[0.127 ln 0.127 + 0.873 ln 0.873] ≈ 0.38: es el piso a superar en el caso de la cooperativa (con clases balanceadas sería ln 2 ≈ 0.693).

## 5 · Descenso de gradiente

El descenso de gradiente da pasos en la dirección opuesta al gradiente, y el tamaño del paso está limitado por la curvatura de la pérdida.

```
\boldsymbol\theta_{t+1} = \boldsymbol\theta_t -
\eta\,\nabla_{\boldsymbol\theta}\mathcal{L}(\boldsymbol\theta_t)
```

**Analogía: bajar del Cotopaxi con neblina.** No ves el valle; solo sientes la pendiente bajo tus pies. Das un paso hacia donde más baja. Pasos cortos llegan tarde; pasos enormes saltan al otro lado de la quebrada.

**Por qué −∇ℒ.** Por Taylor de primer orden, ℒ(θ + Δ) ≈ ℒ(θ) + ∇ℒ(θ)ᵀΔ. Entre todos los Δ con ‖Δ‖ = ε, Cauchy–Schwarz da ∇ℒᵀΔ ⩾ −ε‖∇ℒ‖, con igualdad en Δ = −ε∇ℒ/‖∇ℒ‖: la dirección de mayor descenso local. Con paso η, la pérdida baja aproximadamente η‖∇ℒ‖².

**Lema del descenso.** Si el gradiente es L-Lipschitz (curvatura acotada por L):

```
\mathcal{L}(\boldsymbol\theta - \eta\nabla\mathcal{L}) \le \mathcal{L}
(\boldsymbol\theta) - \eta\left(1 - \frac{L\eta}
{2}\right)\lVert\nabla\mathcal{L}\rVert^2
```

La pérdida baja en cada paso si $0 < \eta < 2/L$, y la cota de descenso es máxima con $\eta = 1/L$.

**El caso cuadrático.** Cerca de un mínimo, $\mathcal{L} \approx \tfrac{1}{2}\theta^{\top} H \theta$. En la base de vectores propios de H, cada componente evoluciona como $\theta_{i,t} = (1 - \eta\lambda_i)^t \, \theta_{i,0}$. Converge si y solo si $|1 - \eta\lambda_i| < 1$ para todo i, es decir $0 < \eta < 2/\lambda_{max}$. La dirección más curva limita el paso y la menos curva fija la velocidad; el número de condición $\kappa = \lambda_{max}/\lambda_{min}$ mide cuánto zigzag habrá.

| η (con λ = 1 y 12) | Comportamiento |
|---|---|
| 0.02 | Converge muy lento |
| 0.15 | Converge en zigzag: $1 - 0.15 \cdot 12 = -0.8$ |
| $0.172 > 2/12 \approx 0.167$ | Diverge |

## 6 · Perceptrón multicapa (MLP) y aproximación universal

Un MLP apila capas de neuronas con activaciones no lineales; cada capa construye una representación nueva que la siguiente usa como entrada.

```
\mathbf{a}^{(0)} = \mathbf{x},\qquad \mathbf{z}^{(l)} =
W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},\qquad \mathbf{a}^{(l)} =
\varphi^{(l)}\bigl(\mathbf{z}^{(l)}\bigr),\qquad \hat{\mathbf{y}} =
\mathbf{a}^{(L)}
```

$W^{(l)}$ tiene tamaño $n_l \times n_{l-1}$ (filas = neuronas de la capa, columnas = entradas que reciben). Una capa densa tiene $n_{l-1} \cdot n_l + n_l$ parámetros. En el caso de la cooperativa (28 entradas → 32 → 16 → 1): 928 + 528 + 17 = 1 473 parámetros.

**Convención de código.** NumPy y Keras procesan lotes con filas = ejemplos: $Z = A \cdot W + \mathbf{b}$ con W de tamaño $n_{l-1} \times n_l$. Es la misma cuenta, transpuesta.

**Por qué resuelve XOR.** Cada neurona oculta traza su propio hiperplano; la salida combina esas regiones. Con dos neuronas ocultas que detectan "al menos una entrada encendida" y "ambas encendidas", la salida calcula la primera menos la segunda.

**Teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991; Leshno et al., 1993). Si φ es continua y no polinomial, para toda f continua en un compacto $K \subset \mathbb{R}^d$ y todo $\varepsilon > 0$ existen N y parámetros tales que:

```
\sup_{\mathbf{x}\in K}\;\Bigl|\,f(\mathbf{x}) -
\sum_{i=1}^{N}\alpha_i\,\varphi\bigl(\mathbf{w}_i^\top\mathbf{x} +
b_i\bigr)\Bigr| < \varepsilon
```

La letra pequeña importa: el teorema garantiza que la red existe, no que el descenso de gradiente la encuentre, ni cuántas neuronas o datos hacen falta, ni que generalice. Analogía: la vía a Baños tiene curvas, pero se puede trazar con tramos rectos; una red ReLU es lineal por tramos y cada neurona agrega un posible quiebre.

## 7 · Retropropagación

La retropropagación calcula todas las derivadas $\partial\mathcal{L}/\partial\theta$ con un solo recorrido hacia atrás, aplicando la regla de la cadena capa por capa (Rumelhart, Hinton y Williams, 1986).

**Analogía: la cadena del cacao fino de aroma.** Finca → fermentación → tostado → barra → catadora. Si la barra sale amarga, la responsabilidad de la finca es el producto de cuánto influye cada etapa en la siguiente; si un grano alimenta varias barras, las culpas por cada ruta se suman.

**Definición clave.** El error de la capa $l$ es $\delta^{(l)} \equiv \partial\mathcal{L}/\partial\mathbf{z}^{(l)}$.

**Las cuatro ecuaciones** ($\odot$ = producto elemento a elemento; notación de Nielsen, 2015):

```
\begin{aligned}
&\text{(BP1)}\quad \boldsymbol\delta^{(L)} = \nabla_{\mathbf{a}}\mathcal{L}
\odot \varphi'\bigl(\mathbf{z}^{(L)}\bigr) \qquad (\text{BCE + sigmoide: }
\boldsymbol\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y})\\
&\text{(BP2)}\quad \boldsymbol\delta^{(l)} =
\bigl(W^{(l+1)\top}\boldsymbol\delta^{(l+1)}\bigr) \odot
\varphi'\bigl(\mathbf{z}^{(l)}\bigr)\\
&\text{(BP3)}\quad \frac{\partial \mathcal{L}}{\partial W^{(l)}} =
\boldsymbol\delta^{(l)}\,\mathbf{a}^{(l-1)\top}\\
&\text{(BP4)}\quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} =
\boldsymbol\delta^{(l)}
\end{aligned}
```

**Derivación de BP2.** $z_j^{(l)}$ influye en $\mathcal{L}$ solo a través de las $z_k^{(l+1)}$ de la capa siguiente:

```
\delta_j^{(l)} = \sum_k \frac{\partial \mathcal{L}}{\partial
z_k^{(l+1)}}\,\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}},\qquad
z_k^{(l+1)} = \sum_j W_{kj}^{(l+1)}\,\varphi\bigl(z_j^{(l)}\bigr) +
b_k^{(l+1)}
```

Como $\partial z_k^{(l+1)}/\partial z_j^{(l)} = W_{kj}^{(l+1)}\,\varphi'(z_j^{(l)})$, queda $\delta_j^{(l)} = \varphi'(z_j^{(l)})\,\sum_k W_{kj}^{(l+1)}\,\delta_k^{(l+1)}$, que es la componente j de BP2. La transpuesta aparece porque hacia atrás necesitamos todo lo que sale de la neurona j (una columna de W), no lo que le entra (una fila).

**Derivación de BP3 y BP4.** $z_j^{(l)} = \sum_i W_{ji}^{(l)}\,a_i^{(l-1)} + b_j^{(l)}$, así que $\partial\mathcal{L}/\partial W_{ji}^{(l)} = \delta_j^{(l)}\,a_i^{(l-1)}$ y $\partial\mathcal{L}/\partial b_j^{(l)} = \delta_j^{(l)}$.

**Algoritmo (un mini-batch).**

1. Forward: calcular y guardar $\mathbf{z}^{(l)}$ y $\mathbf{a}^{(l)}$ de cada capa.
2. BP1 en la salida.
3. BP2 desde $l = L-1$ hasta 1.
4. BP3 y BP4 en cada capa; promediar sobre el lote.
5. Actualizar con el optimizador (sección 10).

**Forma por lotes** (filas = ejemplos, B ejemplos): $\Delta^{(L)} = (\hat{Y} - Y)/B$, $\nabla W^{(l)} = A^{(l-1)\top}\Delta^{(l)}$, $\nabla\mathbf{b}^{(l)} = \mathbf{1}^{\top}\Delta^{(l)}$ y $\Delta^{(l-1)} = (\Delta^{(l)}\,W^{(l)\top}) \odot \varphi'(Z^{(l-1)})$. Así está implementado en el NB03 (sección 6).

**Costo.** Con diferencias finitas, el gradiente exige 2P evaluaciones de la red (P parámetros: 2 946 para nuestro MLP). La diferenciación automática en modo reverso cuesta del orden de un forward, sin importar P, a cambio de guardar las activaciones. Las diferencias finitas se usan solo para verificar (*gradient check*): error relativo $< 10^{-7}$ es excelente, $> 10^{-4}$ sospechoso. Con ReLU, si algún z vale exactamente 0 (por ejemplo, sesgos en cero y una capa muerta) el check puede dar un falso positivo; se repite con sesgos pequeños distintos de cero.

## 8 · Ejemplo resuelto: red 2-2-1

Un solo paso de gradiente con $\eta = 0.5$ sube la probabilidad predicha de 0.557 a 0.654 y baja la pérdida de 0.585 a 0.424. Los números son los de las láminas 51–55 y los del NB02 (partes A a D); TensorFlow los reproduce con diferencia de $5.6 \times 10^{-17}$.

**Datos.** x = (1.0, 0.5): antigüedad como socio y carga de la cuota, escaladas; y = 1 (pagó a tiempo; ojo, al revés que mora_30d). $W^{(1)} = [[0.5, -0.3], [0.2, 0.8]]$, $b^{(1)} = (0.1, -0.1)$, $W^{(2)} = [0.7, -0.4]$, $b^{(2)} = 0.05$. Activaciones sigmoide y pérdida BCE.

| Paso | Cálculo | Resultado |
|---|---|---|
| Forward capa oculta | $z_1 = 0.5\cdot1.0 - 0.3\cdot0.5 + 0.1$; $z_2 = 0.2\cdot1.0 + 0.8\cdot0.5 - 0.1$ | $z^{(1)} = (0.45, 0.50)$; $a^{(1)} = (0.6106, 0.6225)$ |

| Paso | Cálculo | Resultado |
|---|---|---|
| Forward salida | $z^{(2)} = 0.7 \cdot 0.6106 - 0.4 \cdot 0.6225 + 0.05$ | $z^{(2)} = 0.2285$; $\hat{y} = 0.5569$ |
| Pérdida | $-\log 0.5569$ | 0.5854 |
| BP1 | $\delta^{(2)} = \hat{y} - y$ | −0.4431 |
| BP3–4 salida | $\delta^{(2)}\, a^{(1)T}$ | $\nabla W^{(2)} = (-0.2706, -0.2758)$; $\nabla b^{(2)} = -0.4431$ |
| BP2 | $(W^{(2)T}\delta^{(2)}) \odot \sigma'(z^{(1)}) = (-0.3102, 0.1773) \odot (0.2378, 0.2350)$ | $\delta^{(1)} = (-0.0738, 0.0417)$ |
| BP3–4 oculta | $\delta^{(1)}\, x^T$ | $\nabla W^{(1)} = [[-0.0738, -0.0369], [0.0417, 0.0208]]$; $\nabla b^{(1)} = \delta^{(1)}$ |
| Actualización ($\eta = 0.5$) | $\theta \leftarrow \theta - \eta \nabla \theta$ | $W^{(2)} = (0.8353, -0.2621)$; $b^{(2)} = 0.2716$; $W^{(1)} = [[0.5369, -0.2816], [0.1792, 0.7896]]$; $b^{(1)} = (0.1369, -0.1208)$ |
| Verificación | Nuevo forward | $\hat{y} = 0.6542$; pérdida = 0.4243 |

**Lectura.** La neurona oculta 1 empuja hacia "paga" (peso +0.7), así que su gradiente es negativo y sus pesos suben; la neurona 2 empuja en contra (−0.4) y sus pesos bajan. Al pasar de la salida a la capa oculta, el error se multiplica por $\sigma' \approx 0.24$: el desvanecimiento del gradiente ya es visible en una sola capa.

## 9 · Inicialización, desvanecimiento y explosión

Una red profunda solo es entrenable si la señal (hacia adelante) y el gradiente (hacia atrás) conservan su escala de capa en capa; la inicialización se elige para lograrlo.

**El problema.** Aplicando BP2 recursivamente, con $D^{(l)} = \mathrm{diag}(\varphi'(\mathbf{z}^{(l)}))$:

```
\boldsymbol\delta^{(1)} = D^{(1)}W^{(2)\top}\,D^{(2)}W^{(3)\top}\cdots D^{(L-1)}W^{(L)\top}\,\boldsymbol\delta^{(L)}
```

Si esos factores achican, el gradiente se desvanece y las primeras capas no aprenden; si agrandan, explota y la pérdida da NaN. Con sigmoide ($\varphi' \leqslant 1/4$), diez capas pueden reducir la señal a $(1/4)^{10} \approx 10^{-6}$. Analogía: el teléfono dañado, donde un mensaje que pasa por 20

personas se apaga o se exagera.

**Simetría.** Si todos los pesos de una capa empiezan iguales, todas sus neuronas calculan lo mismo y reciben el mismo gradiente, así que siguen idénticas para siempre. Analogía: una banda de pueblo donde todos tocan la misma nota. Los pesos deben ser aleatorios; los sesgos sí pueden empezar en cero.

**Escala.** Si entradas y pesos son independientes y de media cero, $\mathrm{Var}(z) = n_{in} \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(x)$. Para conservar la varianza:

| Esquema | Var(w) | Para |
|---|---|---|
| Xavier/Glorot (Glorot y Bengio, 2010) | $2 / (n_{in} + n_{out})$ | tanh, sigmoide |
| He/Kaiming (He et al., 2015) | $2 / n_{in}$ | ReLU (anula en promedio la mitad de las activaciones) |

En un experimento con 15 capas ReLU de ancho 256: con desviación 0.01 la señal cae a ~$10^{-15}$, con 0.10 crece y con He se mantiene estable. En Keras, `Dense` usa `glorot_uniform` por defecto; con ReLU conviene `kernel_initializer="he_normal"`.

## 10 · Optimizadores: SGD, momentum y Adam

Ningún optimizador gana siempre: momentum bien ajustado gana en problemas cuadráticos, y Adam es la opción más robusta sin ajuste fino.

**SGD con mini-batches.** Con un subconjunto aleatorio $\mathcal{B}$ de tamaño B, el gradiente estimado $\hat{g} = (1/B) \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta)$ es insesgado, $E[\hat{g}] = \nabla \mathcal{L}(\theta)$, y su error cuadrático medio decrece como 1/B. Analogía: el gradiente completo es un censo; el mini-batch, una encuesta por muestreo. Una época es una pasada por todos los datos: con N = 4 800 y B = 64 son 75 iteraciones.

**Momentum** (Polyak, 1964):

$$\mathbf{v}_t = \beta\,\mathbf{v}_{t-1} + \mathbf{g}_t, \qquad \boldsymbol\theta_{t+1} = \boldsymbol\theta_t - \eta\,\mathbf{v}_t, \qquad \mathbf{v}_t = \sum_{k\ge 0}\beta^k\,\mathbf{g}_{t-k}$$

En una dirección constante el paso efectivo es ≈ η/(1 − β), diez veces η con β = 0.9; en direcciones que cambian de signo los gradientes se cancelan y el zigzag baja. Analogía: un ciclista bajando de Baños a Puyo, a quien la inercia lleva por las rectas sin frenar en cada bache.

**Adam** (Kingma y Ba, 2015):

$$
\begin{aligned}
\mathbf{m}_t &= \beta_1\mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, &
\mathbf{s}_t &= \beta_2\mathbf{s}_{t-1} + (1-\beta_2)\,\mathbf{g}_t\odot\mathbf{g}_t\\
\hat{\mathbf{m}}_t &= \mathbf{m}_t/(1-\beta_1^t), & \hat{\mathbf{s}}_t &= \mathbf{s}_t/(1-\beta_2^t)\\
\boldsymbol\theta_{t+1} &= \boldsymbol\theta_t - \eta\,\hat{\mathbf{m}}_t \big/ \bigl(\sqrt{\hat{\mathbf{s}}_t} + \epsilon\bigr)
\end{aligned}
$$

m da la dirección (momentum); s da la escala de cada parámetro, que avanza con más cautela si su gradiente es grande o ruidoso. La corrección de sesgo compensa que m y s empiezan en cero: en t = 1, $m_1 = 0.1\, g_1$ y dividir por 1 − 0.9 recupera $g_1$. Valores por defecto en Keras: $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-7}$. AdamW (Loshchilov y Hutter, 2019) separa el *weight decay* del paso adaptativo. Analogía: un chofer en la Panamericana que acelera en las rectas estables y frena donde la vía es irregular.

**Comparación en un valle con κ = 100** (60 pasos):

| Optimizador | Configuración | Pérdida final |
|---|---|---|
| Momentum | η = 0.06, β = 0.7 (cercanos al óptimo teórico para κ = 100) | $4.3 \times 10^{-8}$ |
| Adam | η = 0.1, betas por defecto | $5.1 \times 10^{-2}$ |
| SGD | $\eta = 0.038$ (justo bajo $2/\lambda_{max} = 0.04$) | $4.0 \times 10^{-1}$ |

**Receta de partida.** Adam con $\eta = 10^{-3}$, B entre 32 y 128, ReLU + He, entradas estandarizadas. Prueba de cordura: el modelo debe poder sobreajustar 32 ejemplos; si no, hay un error de implementación.

# 11 · De la teoría a Keras/TensorFlow

Cada línea de Keras corresponde a un concepto de estos apuntes; `model.fit()` ejecuta el ciclo forward → retropropagación → actualización por mini-batches y épocas.

| Código (Keras 3 / TensorFlow 2) | Concepto |
|---|---|
| `keras.Input(shape=(28,))` | $n_0 = 28$ entradas |
| `layers.Dense(32, activation="relu", kernel_initializer="he_normal")` | $28 \times 32 = 896$ pesos (Keras guarda W como entradas × salidas; aquí $W^{(1)}$ es 32 × 28), ReLU, inicialización He |
| `layers.Dense(1, activation="sigmoid")` | $\hat{y} = \sigma(z^{(L)})$ |
| `loss="binary_crossentropy"` | BCE (sección 4) |
| `keras.optimizers.Adam(1e-3)` / `SGD(0.05, momentum=0.9)` | Sección 10 |
| `batch_size=64`, `epochs=40` | Mini-batch y épocas |
| `tf.GradientTape()` + `tape.gradient(loss, vars)` | Retropropagación automática (modo reverso) |
| `optimizer.apply_gradients(zip(grads, vars))` | $\theta \leftarrow \theta - \eta \cdot$(regla del optimizador) |

El bucle propio con `GradientTape`, que es lo que `fit()` hace por dentro:

```
for epoch in range(epochs):
    for xb, yb in dataset:                                  # mini-batches
        with tf.GradientTape() as tape:
            loss = loss_fn(yb, model(xb, training=True))      # forward
(recorded)
        grads = tape.gradient(loss, model.trainable_variables)        #
backprop
        optimizer.apply_gradients(zip(grads, model.trainable_variables))  #
update
```

Keras 3 es una reescritura multi-backend: la misma API corre sobre JAX, TensorFlow o PyTorch (y OpenVINO para inferencia). En Colab viene con TensorFlow. Un tensor es un arreglo con forma y tipo: (28,) es un socio, (64, 28) un mini-batch de socios, (64, 224, 224, 3) un lote de imágenes; el primer eje casi siempre es el batch.

# 12 · Autoevaluación

Ocho preguntas del estilo del test de la semana; intenta responderlas antes de leer las respuestas.

1. Con **w** = (2, −1) y b = 0, ¿de qué lado del hiperplano queda **x** = (1, 3)?
2. Demuestra que con BCE y salida sigmoide, $\partial \mathcal{L}/\partial z = \hat{y} - y$.
3. ¿Por qué una red de 50 capas sin activaciones no puede separar XOR?
4. En una cuadrática con valores propios 2 y 40, ¿cuál es el mayor $\eta$ que garantiza convergencia del descenso de gradiente?
5. Una neurona ReLU tuvo $z < 0$ para todos los ejemplos de un lote. ¿Qué gradiente reciben sus pesos de entrada?
6. ¿Cuántos parámetros tiene un MLP 10-64-32-1?
7. ¿Por qué Adam divide $m_t$ por $1 - \beta_1^t$?
8. Un modelo de fraude tiene 98 % de accuracy con 2 % de fraudes. ¿Qué métrica pedirías antes de celebrar y por qué?

**Respuestas**

1. $z = 2 \cdot 1 - 1 \cdot 3 = -1 < 0$: lado negativo.
2. $\partial \mathcal{L}/\partial \hat{y} = -y/\hat{y} + (1 - y)/(1 - \hat{y})$ y $\partial \hat{y}/\partial z = \hat{y}(1 - \hat{y})$; el producto es $-y(1 - \hat{y}) + (1 - y)\hat{y} = \hat{y} - y$.
3. La composición de funciones afines es afín: toda la red equivale a un solo hiperplano, y XOR no es linealmente separable.
4. $\eta < 2/\lambda_{max} = 2/40 = 0.05$.
5. Cero: $\varphi'(z) = 0$ hace $\delta = 0$ y, por BP3, el gradiente de sus pesos es 0. Si le pasa con todos los datos, es una neurona muerta.
6. $(10 \cdot 64 + 64) + (64 \cdot 32 + 32) + (32 \cdot 1 + 1) = 704 + 2\,080 + 33 = 2\,817$.
7. Porque m arranca en 0 y los primeros promedios están sesgados hacia cero; la división corrige ese sesgo (en $t = 1$ recupera $g_1$).
8. Predecir siempre "no fraude" ya da 98 %. Hay que mirar PR-AUC (su línea base es 0.02), ROC-AUC y recall/precisión en un umbral de negocio.

# Referencias

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems, 2*(4), 303–314.

Glorot, X., y Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. En *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics* (pp. 249–256). PMLR.

He, K., Zhang, X., Ren, S., y Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En *Proceedings of the IEEE International Conference on Computer Vision* (pp. 1026–1034).

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks, 4*(2), 251–257.

Isola, P., Beery, S., y Bernstein, J. (2023). *6.S898 Deep learning* [Curso]. MIT. https://phillipi.github.io/6.s898/

Kingma, D. P., y Ba, J. (2015). Adam: A method for stochastic optimization. En *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

Leshno, M., Lin, V. Y., Pinkus, A., y Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks, 6*(6), 861–867.

Loshchilov, I., y Hutter, F. (2019). Decoupled weight decay regularization. En *International Conference on Learning Representations*. https://arxiv.org/abs/1711.05101

Minsky, M., y Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.

Nielsen, M. A. (2015). *Neural networks and deep learning*. Determination Press. http://neuralnetworksanddeeplearning.com/chap1.html

Novikoff, A. B. J. (1962). On convergence proofs on perceptrons. En *Proceedings of the Symposium on the Mathematical Theory of Automata* (Vol. 12, pp. 615–622). Polytechnic Institute of Brooklyn.

Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics, 4*(5), 1–17.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review, 65*(6), 386–408.

Rumelhart, D. E., Hinton, G. E., y Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536.

Zhang, A., Lipton, Z. C., Li, M., y Smola, A. J. (s. f.). *Dive into deep learning*. Cambridge University Press. https://d2l.ai/chapter_multilayer-perceptrons/index.html